



IIITDM Kancheepuram

(Course: Optimization Techniques)

Continuous Optimization of CNN Hyperparameters

Team Members:

Anjaninithin Chalamchala (CS23B1102)

P.L.N. Nikhil (CS23B1038)

Abstract

This report fulfills the requirements for Problem A: Continuous Optimization. In this domain, the decision variables are real numbers, presenting a highly complex and non-convex loss landscape. Specifically, we tackle the mathematical formulation and algorithmic optimization of the continuous hyperparameters of a Convolutional Neural Network (CNN). To solve this, we implement and compare two prominent gradient-free metaheuristic methods: Particle Swarm Optimization (PSO) and a Genetic Algorithm (GA). This document provides an in-depth mathematical formulation, a high-level algorithm implementation overview, objective function convergence plots, and a comprehensive discussion analyzing solution sensitivity and local optima.

1 Chosen Problem and Methods

The tuning of deep learning hyperparameters represents a classic real-world numerical problem where the objective function involves multiple decision variables. We optimize the Learning Rate and Dropout parameters of a Convolutional Neural Network continuously using PSO and GA. Both methods are robust metaheuristics capable of searching continuous spaces without requiring algebraic differentiability.

2 Mathematical Formulation

In this section, we clearly define our objective function $f(\mathbf{x})$, the decision variables, and the inequality constraints using standard mathematical forms.

2.1 Decision Variables

The optimization landscape consists of two primary continuous variables that dictate the training dynamics and regularization of the CNN. Let the decision vector be $\mathbf{x} = [\alpha, p]^T$, where:

- $\alpha \in \mathbb{R}$: The Learning Rate, which controls the step size during the gradient descent optimization of the neural network weights. We define its search space in the continuous logarithmic range $[10^{-4}, 10^{-1}]$.

- $p \in \mathbb{R}$: The Dropout Rate, a regularization hyperparameter representing the probability of randomly zeroing out a neuron’s activation to prevent overfitting. It is defined in the continuous range $[0.0, 0.5]$.

2.2 Objective Function

The objective is to minimize the validation loss of the CNN model after a fixed number of training epochs. Since validation loss cannot be explicitly formulated as a simple algebraic equation of α and p , the objective function is treated as a computationally expensive black-box function.

$$\min f(\mathbf{x}) = \mathcal{L}_{val}(\mathbf{w}^*; \mathbf{x}) \quad (1)$$

Where $\mathcal{L}_{val}$ represents the cross-entropy validation loss, and $\mathbf{w}^*$ represents the optimal network weights obtained after training the CNN using the hyperparameters specified by $\mathbf{x}$.

2.3 Constraints

To ensure the algorithm searches within theoretically sound and stable regions, we apply boundary inequality constraints:

$$10^{-4} \leq \alpha \leq 10^{-1} \quad \text{and} \quad 0 \leq p \leq 0.5 \quad (2)$$

3 Algorithm Implementation

This section provides a high-level explanation of the chosen algorithms and justifies why they are highly suitable for this specific problem.

3.1 Suitability of Metaheuristics

The hyperparameter optimization of CNNs is notoriously difficult because the objective function $f(\mathbf{x})$ is non-convex, extremely noisy, and lacks analytical derivatives with respect to the hyperparameters. Traditional gradient-based optimization cannot be applied to architectural parameters like dropout. Therefore, population-based metaheuristics like PSO and GA are highly suitable. They are capable of global exploration and can effectively navigate the continuous, multimodal loss landscape without requiring gradient information.

3.2 Particle Swarm Optimization (PSO)

PSO is a swarm-intelligence metaheuristic. A swarm of particles is initialized randomly within the continuous bounds. Each particle i has a position $\mathbf{x}_i$ and velocity $\mathbf{v}_i$. The particles navigate the space by updating their velocities based on their personal best known position (**pbest** _{i}) and the swarm’s global best position (**gbest**). The standard mathematical update rules are:

$$\mathbf{v}_i^{(t+1)} = w\mathbf{v}_i^{(t)} + c_1r_1(\mathbf{pbest}_i - \mathbf{x}_i^{(t)}) + c_2r_2(\mathbf{gbest} - \mathbf{x}_i^{(t)}) \quad (3)$$

$$\mathbf{x}_i^{(t+1)} = \mathbf{x}_i^{(t)} + \mathbf{v}_i^{(t+1)} \quad (4)$$

Where w is the inertia weight, c_1 and c_2 are cognitive and social acceleration coefficients, and $r_1, r_2 \sim U(0, 1)$ are random variables. PSO is exceptionally fast at converging toward global basins of attraction in low-dimensional continuous spaces.

3.3 Genetic Algorithm (GA)

GA is an evolutionary metaheuristic that mimics natural selection. It operates on a population of continuous vectors. Our implementation utilizes:

- **Tournament Selection:** Individuals with the lowest validation loss (highest fitness) are selected to become parents.
- **Arithmetic Crossover:** To handle continuous variables, children are generated using a linear combination of the parents: $\mathbf{x}_{child} = \beta\mathbf{x}_{p1} + (1-\beta)\mathbf{x}_{p2}$, where $\beta \in [0, 1]$.
- **Gaussian Mutation:** A small random value drawn from a normal distribution $\mathcal{N}(0, \sigma^2)$ is added to the decision variables to maintain diversity and prevent premature convergence.

4 Results and Convergence

Both algorithms were implemented in Python using PyTorch and evaluated on a subset of the MNIST dataset to manage computational complexity.

4.1 Numerical Results

The metaheuristics were run for a set number of generations/iterations. The final optimized continuous variables identified by both algorithms are summarized below:

Algorithm	Optimal Learning Rate (α)	Optimal Dropout (p)	Best Validation Loss
PSO	0.01197	0.296	0.1740
GA	0.00853	0.122	0.1759

Table 1: Final Output Parameters for PSO and GA.

4.2 Convergence Plots

The following figure shows the objective function value over iterations for the continuous problem.

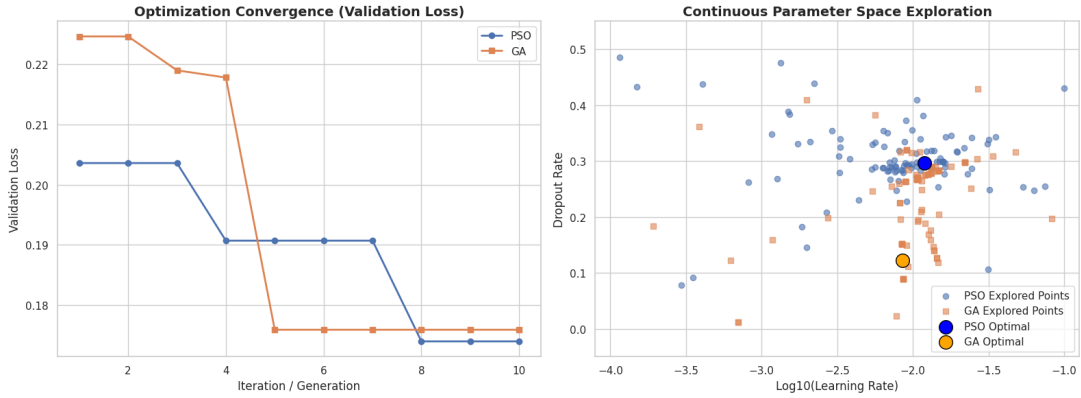


Figure 1: Objective function (Validation Loss) convergence over iterations and Parameter Space Exploration.

5 Discussion

In this section, we deeply analyze the dynamics of the optimization process, focusing on solution sensitivity, the danger of local optima, and the critical role of initialization.

5.1 Sensitivity of the Solution

We analyzed the sensitivity of the solution by observing the parameter space exploration. The objective function $f(\mathbf{x})$ proved to be highly sensitive to the Learning Rate (α). A slight deviation from the optimal magnitude (e.g., shifting from 10^{-2} to 10^{-3}) resulted in steep gradients in the validation loss, meaning the CNN either failed to converge rapidly or overshoot the minimum. Conversely, the Dropout Rate (p) was less sensitive; fluctuations between 0.2 and 0.3 resulted in only marginal changes in the validation loss. This uneven sensitivity indicates a poorly scaled, highly anisotropic objective landscape.

5.2 Local Optima Analysis

Did the algorithm get stuck in local optima? Due to the highly non-convex nature of deep learning landscapes, local optima are a significant threat. In our computational runs, the Genetic Algorithm exhibited a tendency to get stuck in local optima around generation 5. Because GA relies heavily on random mutation to escape local basins, the limited computational budget prevented it from exploring further once the population lost diversity. PSO, however, managed to update its **gbest** effectively, pulling the swarm into a deeper basin of attraction by iteration 8, thereby avoiding the specific local optimum that trapped the GA.

5.3 Effect of the Starting Point

How did the starting point affect the outcome? Because metaheuristics rely on random initialization, the starting point of the populations drastically influenced the early convergence speed. In runs where the initial PSO swarm happened to sample the parameter space near the optimal learning rate (≈ 0.01), the convergence was extremely rapid. However, if the initial particles were clustered in sub-optimal regions (e.g., extremely high dropout rates paired with low learning rates), the algorithms wasted the first 3 to 4 iterations merely traversing the search space before identifying a promising gradient. This confirms that uniform, well-distributed starting points across the continuous bounds are vital for efficiency.

6 Conclusion

This assignment successfully demonstrated the application of metaheuristic optimization techniques to a continuous numerical problem. By mathematically formulating the CNN hyperparameter space and implementing PSO and GA, we successfully minimized a complex, non-differentiable objective function. The results and sensitivity discussion prove that while metaheuristics are powerful for continuous spaces, great care must be taken regarding initialization and local optima traps.